CHOREO

算子专用编程语言介绍系列

编程篇之零：编写第一个 *Choreo* 算子

Talk is Cheap. Show Me the Code!

在之前的资讯中，我们为大家介绍了 Choreo 的种种优点。这次让我们编写一个完整 Choreo 的程序来看看它是如何简化算子编程的。

```
__co__ auto ele_add(s32 [6, 17, 64] lhs, s32 [6, 17, 64] rhs) {
  s32 [lhs.span] output;

  parallel p by 6
    foreach index in [17, 4] {
      lf = dma.copy.async lhs.chunkat(p, index) => local;
      rf = dma.copy.async rhs.chunkat(p, index) => local;
      wait lf, rf;

      local s32 [lf.span] l1_out;

      foreach i in [l1_out.span]
        l1_out.at(i) = lf.data.at(i) + rf.data.at(i);

      dma.copy l1_out => output.chunkat(p, index);
    }

  return output;
}

int main() {
  choreo::s32 a[6][17][64] = {0};
  choreo::s32 b[6][17][64] = {0};

  auto a_data = choreo::make_spanview<3, choreo::s32>((int*)a, {6, 17, 64});
  auto b_data = choreo::make_spanview<3, choreo::s32>((int*)b, {6, 17, 64});
  a_data.fill_random(-10, 10);
  b_data.fill_random(-10, 10);

  auto res = ele_add(a_data, b_data);

  for (size_t i = 0; i < res.shape()[0]; ++i)
    for (size_t j = 0; j < res.shape()[1]; ++j)
      for (size_t k = 0; k < res.shape()[2]; ++k)
        choreo::choreo_assert(a[i][j][k] + b[i][j][k] == res[i][j][k],
                              "values are not equal.");
  printf("Test Passed\n");
  return 0;
}
```

以上代码是一个完整的 *Choreo-C++* 程序。这个程序由两部分构成：*Tileflow* 和 *Host*。*Tileflow* 代码是以 `__co__` 为前缀的函数构成的。可以看到，与 C++ 函数不同地是，*Tileflow* 函数的参数可以是**有形状的张量**。而形状则由 `[6, 17, 64]` 这样的维度值列表确定。

形状是 Choreo 最为核心的部分，是 Choreo 能够免去程序员显式编写地址下标运算，实现代码安全的基础。不过由于 C++ 程序没有对张量形状的描述，因此在 *Host* 程序（这里的`main`函数）中调用 *Tileflow* 函数的时候，需要通过 Choreo 的 API `choreo::make_spanview`对张量数据进行形状标识，并传入 `ele_add` 函数进行调用。

在 *Tileflow* 函数 `ele_add` 中，我们将输入数据拆分为小块，并在不同的执行线程上迭代地、异步地搬入 L1 存储，以进行逐元素加法。运算完成后结果被依序搬出。具体请参见代码逐行解析。

Tileflow 代码逐行解析

- **函数定义**：`ele_add` 是一个 Choreo Tileflow 函数，它接受两个三维张量 `lhs` 和 `rhs` 作为输入，返回值则需要编译器推导得出。
- **并行化**：`parallel p by 6` 表示其中的代码有 6 个并行的**执行线程**，这里可以认为我们启动了 6 个 SIP 来处理其中的代码，每个 SIP 有唯一的**线程标识** `p`。
- **数据分块（Tiling）与迭代**：`foreach index in [17, 4]` 设定两个切分值 17,4，同时创建两层循环，循环的**迭代变量**为`index`。
- **异步数据移动**：两条`dma.copy.async`语句使用 DMA/DTE 将数据从全局内存异步传输到本地内存。`rhs.chunkat(p, index)` 表达式则由线程标识以及循环迭代变量确定的数据分块。
- **同步等待**：`wait lf, rf;` 等待两个异步数据传输完成。
- **本地计算**：在本地内存中创建一个临时数组 `l1_out`，并逐元素相加搬入的数据。
- **结果同步写回**：`dma.copy` 语句将计算结果从本地内存写回到全局内存中的分块位置中。
- **返回结果**：最终返回计算完成的 `output` 数组。

CHOREO

生成等效的 Topscc 源码

在 Choreo 现在的工作流程中，会将 *Choreo-C++* 代码生成为**等效 *topscc* 代码**，并调用 *topscc* 编译器实施最终的编译。而且程序员可以使用`-es`选项来获得该等效的 *topscc* 代码输出：

```
choreo -es ele.co -o ele.cpp
```

这样一来，`ele.cpp`可以被程序员修改或借鉴。但这种工作流程需要程序员使用 *topscc* 完成 GCU 平台上的后续编译工作。

欢迎加入 Choreo 服务群，汇报 Choreo 的问题，分享试用体验，以及提出建议

安装 Choreo Pre-Beta Release 来编译运行 Choreo 代码

让我们安装 Choreo 来编译上页中的示例程序。我们可以从http://git.enflame.cn/xiaofeng.guan/choreo/-/packages/66下载 *Choreo Pre-Beta Release* 的 Debian 包进行安装。

```
gxf@docker_dev:~/test$ sudo dpkg -i choreo-1.0.0.pre-beta-Linux.deb
(Reading database ... 103611 files and directories currently installed.)
Preparing to unpack choreo-1.0.0.pre-beta-Linux.deb ...
Unpacking choreo (1.0.0.pre-beta) over (1.0.0.pre-beta) ...
Setting up choreo (1.0.0.pre-beta) ...
```

* 注意：安装 *Choreo* 之前请记得安装*libgmp*，*GCU User Mode Driver and Runtime*，以及*topscc* 编译器。*Choreo Debian* 包会进行相关依赖检查。

Choreo 默认会设定编译目标为 GCU300（Scorpio），且默认输出名为 `a.out` 的可执行文件。下面是最小实施编译的命令：

```
choreo ele.co
```

执行 `a.out`，*Host* 源码中产生的随机数据被搬入 GCU 硬件并并行地相加，而搬回的运算结果则验证为正确：

```
./a.out
Test Passed
```

在常规编译选项上，Choreo 编译器的使用方法与 GCC 编译器类似。比如，编译 GCU210（Dorado）的模块可以采用以下命令：

```
choreo -arch=gcu210 -c ele.co -o ele.o
file ele.o
ele.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not
    stripped
```

可以看到，这里生成的 `ele.o`文件经查验是一个 64 位的可重定位的 ELF 模块。程序员可以使用 `choreo --help` 以及 `choreo --help-hidden` 查询更多可用的编译指令。

深入了解 Choreo

如果想深入了解 Choreo，我们准备了相关的文档、视频教程供您学习：

- 网页教程：http://choreo.enflame.cn/
- 视频教程：https://learning.enflame-tech.com/course/144

参与到 Choreo 的开发中

无论是汇报问题，或是建议新的 Choreo 特性，都可以通过在 Choreo 项目主页创建 issue 的方式达成：http://git.enflame.cn/xiaofeng.guan/choreo/-/issues

Choreo 的**项目主页**在：http://git.enflame.cn/xiaofeng.guan/choreo，其包含了编译使用 Choreo 的说明

CHOREO

- 简化数据搬运编程
- 提高程序安全性
- 支持动态形状

如有需要请联系：

- 官孝峰：*xiaofeng.guan@enflame-tech.com*
- 石恒：*heng.shi@enflame-tech.com*